

目 录

A . 间距调整	/	Brake distance	2
B . 缓存技术	/	Caching technology	4
C . 平滑信号	/	Delivery plan	7
D . 能源划分	/	Energy classification	9
E . 智能调度	/	Fast charging	10
F . 出厂校准	/	Gear calibration	11
G . 服务划分	/	Independent service	13
H . 电池续航	/	Journey endurance	16
I . 电池健康	/	Keep health	18
J . 生产优化	/	Line optimization	20

A . 间距调整 / Brake distance

时间限制： 1.0 秒

【题目描述】

在智能网联汽车队列中，车辆通过 V2V 通信协调行驶，并实时调整与前车的车间距，定义当前车间距为一个正整数，初始间距为 1（单位：米）。一种自适应间距控制策略允许在每一轮控制周期中，对当前间距执行三种原子操作之一：

- **A 操作**（线性增加）：令间距 $+1$ 。
- **B 操作**（倍增扩大）：令间距 $\times 2$ 。
- **C 操作**（回退缩小）：令间距 -1 ，但仅当当前间距严格大于 1 时允许执行（间距始终保持为正整数）。

每执行一次操作计为一步。现给定目标间距 n ，请你构造一条从 1 到 n 的操作序列，使总步数最少。
若存在多条最少步数的序列，输出字典序最小的操作串。

【输入格式】

从标准输入读入数据。
一行一个整数 n ，表示目标车间距。

【输出格式】

输出到标准输出。
第一行输出一个整数 S ，表示最少操作步数。
第二行输出一个长度恰好为 S 的字符串，由 **A**、**B**、**C** 构成，表示最优操作序列。

【样例 1 输入】

1 10

【样例 1 输出】

1 4
2 ABAB

【样例 1 解释】

对于样例输入 10，一种最优序列为：
 $1 \xrightarrow{A} 2 \xrightarrow{B} 4 \xrightarrow{A} 5 \xrightarrow{B} 10$ ，共 4 步，对应操作串 **ABAB**。

【样例 2 输入】

1 15

【样例 2 输出】

1 5

2 ABBBC

【样例 2 解释】

对于样例输入 15，一种最优序列为：

$1 \xrightarrow{A} 2 \xrightarrow{B} 4 \xrightarrow{B} 8 \xrightarrow{B} 16 \xrightarrow{C} 15$ ，共 5 步，对应操作串 ABBBC。

【数据范围与约定】

对于 10% 的数据， $n \leq 10$ 。

对于 30% 的数据， $n \leq 500$ 。

对于 50% 的数据， $n \leq 3000$ 。

对于 100% 的数据， $1 \leq n \leq 10^9$ 。

B . 缓存技术 / Caching technology

时间限制： 1.0 秒

【题目描述】

智能网联汽车中常常需配备端侧大模型以完成各类辅助功能，在大型语言模型的推理过程中，KV Cache 用来缓存历史 token 的 Key 和 Value 张量，以避免重复计算。本题要求你实现一个简化版的 KV 缓存模拟器。该缓存使用全相联映射，通过最近最少使用（LRU）策略决定淘汰顺序，并采用写回方式与主存交互。为了完整描述系统行为，约定如下：

主存：主存可视为一个无限大的存储空间，其中的内容以“键-值”对的形式出现，主存中每个整数键 key 对应一个整数值 $value$ 。系统初始时，对于任意整数 key ，主存中该键的值 $value$ 等于 key 本身。主存中的值只会因为后文定义的“淘汰写回”而改变。

缓存：缓存是比主存小得多的一层存储，用于暂存整数键在主存中对应值的副本。同一整数键在同一时刻至多存在于缓存的一条记录里，记录中的值为该键在缓存侧的当前取值。系统对每个 READ / WRITE 都会先在缓存内按键查找：若查到则走后文的命中流程；否则为缺失，需先从主存得到该键当前的值并按后文约定装入缓存（可能伴随淘汰）。

缓存容量至多 C 个键值对（ C 由输入给定），初始为空。每条缓存记录包含键 key 、值 $value$ ，以及标明是否与主存内容一致的脏标记——被写操作改写过但尚未通过淘汰写回补写主存的条目视为脏（详见下文“脏/干净”与“写回”）。

全相联映射：全相联是与「直接映射 / 组相联」相对而言的一种放置规则，含义为：任意键都可放入缓存中的任意位置；键与缓存内的具体位置之间没有预设映射表。

LRU（最近最少使用）：缓存内维护一个有序序列，一端为 MRU（最近使用），另一端为 LRU（最久未使用）。任意一次访问（读命中、写命中、缺失加载）都会使对应键移动到 MRU 端。缓存满时加载新键，需要淘汰 LRU 端键值对。

写回：写命中时，仅更新缓存值并置脏，不立即更新主存。淘汰时若被淘汰项为脏，则先写回主存再丢弃；若为干净，直接丢弃。

脏/干净

1. 干净：自载入以来（或上次与主存同步后）未被写操作修改，和主存一致。
2. 脏：在缓存中被写操作修改过，和主存不一致。脏数据被淘汰时必须先写回主存。

命中/缺失

1. 命中：访问的键已在缓存中。
2. 缺失：访问的键不在缓存中，需先从主存加载。

系统按顺序处理两类操作：READ key 与 WRITE $key\ value$ 。

读操作 READ key

1. 命中：返回缓存中的值 v ，并将该键移到 MRU 端，脏标记不变。输出：READ $key:Hit,value=v$

2. 缺失：从主存读出当前值 v ，必要时先淘汰 LRU（脏则写回），再将 $\langle \text{key}, v \rangle$ 以干净状态加载到 MRU 端并返回。输出：READ key:Miss,loaded,value= v
写操作 WRITE key value

1. 命中：将缓存中该键更新为 value，标记为脏，并移动到 MRU 端。输出：WRITE key value:Hit,updated

2. 缺失：先执行一次与读缺失相同的加载流程（必要时淘汰），再把该键更新为 value 并置脏（写操作本身也视为访问，仍在 MRU 端）。输出：WRITE key value:Miss,loaded and updated

注意：主存只会在“淘汰脏数据”时被更新。

【输入格式】

从标准输入读入数据。

输入共两行。第一行一个非负整数 C ，表示缓存容量（ $C = 0$ 表示缓存无法存放任何键值对）。

第二行一个完整字符串 S ，表示操作序列。操作之间用分号 ; 分隔。每个操作格式为 READ key 或 WRITE key value，其中 key 与 value 均为 32 位带符号整数。

字符串中允许出现任意数量的空白字符（空格等），它们可能出现在操作首尾、分号前后、以及单词之间。你需要正确解析出所有操作并按顺序执行。

【输出格式】

输出到标准输出。

对每个解析出的操作，按执行顺序输出一行：

1. 读命中：READ key:Hit,value= v

2. 读缺失：READ key:Miss,loaded,value= v

3. 写命中：WRITE key value:Hit,updated

4. 写缺失：WRITE key value:Miss,loaded and updated

其中 key 与 value 为操作中的参数， v 为读操作实际读到的值。

当所有操作处理完后，先输出一个空行，再额外输出缓存最终状态：

- 若缓存非空，输出一行 Cachestate(MRU->LRU):。接着按 MRU 到 LRU 顺序，每行输出一个键值对，格式 key:value。
- 若缓存为空，输出 Cache is empty.。

【样例 1 输入】

```
1 3
2 READ 10; WRITE 20 200; READ 10; READ 30; WRITE 10 100; READ 40;
  READ 20
```

【样例 1 输出】

```
1 READ 10:Miss,loaded,value=10
2 WRITE 20 200:Miss,loaded and updated
3 READ 10:Hit,value=10
4 READ 30:Miss,loaded,value=30
5 WRITE 10 100:Hit,updated
6 READ 40:Miss,loaded,value=40
7 READ 20:Miss,loaded,value=200
8
9 Cachestate(MRU->LRU):
10 20:200
11 40:40
12 10:100
```

【样例 1 解释】

以下给出样例关键过程说明：

- 1 READ 10: 缺失，加载 $\langle 10, 10 \rangle$ （干净），缓存 $[10]$ 。
- 2 WRITE 20 200: 缺失，先加载 $\langle 20, 20 \rangle$ ，再改为 $\langle 20, 200 \rangle$ （脏），缓存 $[20, 10]$ 。
- 3 READ 10: 命中，移到 MRU，缓存 $[10, 20]$ 。
- 4 READ 30: 缺失，加载 $\langle 30, 30 \rangle$ （干净），缓存 $[30, 10, 20]$ 。
- 5 WRITE 10 100: 命中，更新并置脏，缓存 $[10, 30, 20]$ 。
- 6 READ 40: 缺失且缓存满，淘汰 LRU $\langle 20, 200 \rangle$ （脏，写回主存），再加载 $\langle 40, 40 \rangle$ ，缓存 $[40, 10, 30]$ 。
- 7 READ 20: 缺失，主存中 20 已为 200；淘汰 LRU $\langle 30, 30 \rangle$ （干净），加载 $\langle 20, 200 \rangle$ ，缓存 $[20, 40, 10]$ 。

【数据范围与约定】

对于 10% 的数据， $C = 1$ 。

对于全部数据， $0 \leq C \leq 1000$ ， $-2^{31} \leq key, value \leq 2^{31} - 1$ 。操作串长度 $|S| \leq 2 \times 10^5$ 。

C . 平滑信号 / Delivery plan

时间限制： 1.0 秒

【题目背景】

智能网联汽车（Intelligent Connected Vehicle, ICV）在传统车辆基础上融合环境感知、智能决策、协同控制与车路云一体化通信，通过摄像头、激光雷达、毫米波雷达等多源传感器与 V2X 等信息通道，实现对道路、障碍物与其他交通参与者的连续观测。感知链路中的原始量测往往需要在边缘或域控侧做滤波与融合，才能在规划与控制环节形成稳定、可信的输入。

在智能网联汽车系统中，车载传感器（如激光雷达、毫米波雷达）采集到的距离或速度序列常受到环境噪声干扰，影响车辆对环境感知的准确性。滑动均值滤波器是一种常用的去噪工具。定义一个信号是“绝对平滑”的，当且仅当无论用多大的窗口去观察它的局部平均，得到的结果都完全一致。现在，你作为算法工程师，需要用最小的代价将一段充满噪声的信号修改为这种绝对平滑的信号，从而保障自动驾驶决策的稳定性与一致性。

给定一个长度为 n 的整数序列 $A = [A_1, A_2, \dots, A_n]$ 以及一个正整数 k 。

对于窗口长度 w ($1 \leq w \leq k$)，我们使用大小为 w 的一维滑动均值滤波器对序列进行处理：对于所有 $i = 1, 2, \dots, n - w + 1$ ，计算

$$B_i^{(w)} = \frac{1}{w} \sum_{j=0}^{w-1} A_{i+j},$$

得到长度为 $n - w + 1$ 的新序列 $B^{(w)}$ 。

如果对于所有 $w \in [1, k]$ ，滤波后的序列 $B^{(w)}$ 中的每一个值都相等（即 $B^{(w)}$ 是常数序列），那么我们称原序列 A 是平滑的。

你可以任意修改序列 A 中的若干元素（修改后的值可以是任意整数）。请问：最少需要修改多少个元素，才能使得修改后的序列成为平滑序列？

【输入格式】

从标准输入读入数据。

第一行包含两个整数 n, k ，表示序列长度和窗口大小的上限。

第二行包含 n 个整数 $A_1, A_2, \dots, A_n$ ，表示给定的序列。

【输出格式】

输出到标准输出。

输出一个整数，表示最少需要修改的元素数量。

【样例 1 输入】

1 5 3

2 1 2 2 2 3

【样例 1 输出】

1 2

【样例 2 输入】

1 4 2

2 1 2 3 4

【样例 2 输出】

1 3

【样例 3 输入】

1 6 5

2 7 7 7 7 7 7

【样例 3 输出】

1 0

【数据范围与约定】

对于全部测试数据，满足： $1 \leq n \leq 2 \times 10^5$ ， $1 \leq k \leq n$ ， $1 \leq A_i \leq 10^9$ 。

D . 能源划分 / Energy classification

时间限制： 1.0 秒

【题目描述】

某新能源公司运营着一条由 n 个光伏阵列组成的发电带。第 i 个光伏阵列在某一时刻的实际出力（单位：kW）为 a_i 。为了接入电网，公司需要将这 n 个阵列依次划分成恰好 m 个连续的运行周期，每个周期至少包含一个阵列。对于每个运行周期，定义其波动风险值为：

周期长度 $\times$ (周期内的最大出力 - 周期内的最小出力)

整个调度方案的总风险值定义为所有周期中波动风险值的最大值。请你设计一个划分方案，使得总风险值最小，并输出该最小值。

【输入格式】

从标准输入读入数据。

第一行：两个整数 n, m 。

第二行： n 个整数 $a_1, a_2, \dots, a_n$ 表示每个光伏阵列的实时出力

【输出格式】

输出到标准输出。

输出一个整数，表示最小可能的总风险值。

【样例 1 输入】

```
1 6 3
2 4 10 1 6 9 6
```

【样例 1 输出】

```
1 12
```

【数据范围与约定】

对于 40% 的数据， $1 \leq m \leq n \leq 1000$ ， $1 \leq a_i \leq 1000$ 。

对于 100% 的数据， $1 \leq m \leq n \leq 10^5$ ， $1 \leq a_i \leq 10^9$ 。

E . 智能调度 / Fast charging

时间限制： 1.0 秒

【题目背景】

在新型电力系统建设中，你负责运维一座包含三组储能电芯的巨型充电站。由于历史运行损耗不同，目前三组电芯的剩余容量不一致，这限制了电站的整体放电效率。你的任务是通过智能调度，消除电芯间的容量差异，使三者的剩余容量达到一致，以实现电站的最大功率输出。

【题目描述】

给定三个整数 A, B, C ，分别代表三组电芯当前的剩余容量。你可以通过重复执行以下两种调度操作，来增加电芯的容量，最终使 A, B, C 三者完全相等：1. 并联快充：选择任意两组电芯，同时为它们充电，使它们的容量各增加 1 单位。2. 高压直充：选择任意一组电芯，为其进行高压直充，使其容量增加 2 单位。可以证明，通过上述操作的组合，总能使得三组电芯的容量相等。请编写程序，计算使三组电芯容量全部相等所需的最小调度操作次数。

【输入格式】

从标准输入读入数据。
输入的第一行包含三个正整数 A, B, C 。

【输出格式】

输出到标准输出。
输出一个数字，表示所需的最小调度次数。

【样例 1 输入】

1 2 5 4

【样例 1 输出】

1 2

【数据范围与约定】

对于 100% 的数据， $1 \leq A, B, C \leq 10^6$ 。

F . 出厂校准 / Gear calibration

时间限制： 1.0 秒

【题目描述】

随着新能源汽车产业的爆发式增长，退役动力电池的回收利用已成为关键环节。电池中含有四种关键金属元素：**Li**（锂）、**Co**（钴）、**Ni**（镍）、**Mn**（锰）。

你负责运营一家电池回收工厂，现有 m 条不同的回收产线。每条产线最多只能启用一次，启用后会产出一定数量的四种元素，并产生相应的运营成本。集团总部给出了本月的最低回收指标，要求四种元素的最终库存量均不低于目标值。请你计算：满足指标所需的最小总成本。如果无论如何都无法达成目标，请输出 **Impossible**。

【输入格式】

从标准输入读入数据。

输入共 $m + 2$ 行：第 1 行为四个整数 Li_{min} , Co_{min} , Ni_{min} , Mn_{min} ，表示目标产量；第 2 行为整数 m ，表示产线数量；第 3 行至第 $m + 2$ 行每行为 Li_i , Co_i , Ni_i , Mn_i , $cost_i$ ，依次为第 i 条产线的四种元素产量与运营成本。

【输出格式】

输出到标准输出。

输出仅一行：若可达标，输出一个整数最小总成本；否则输出字符串 **Impossible**。

【样例 1 输入】

```
1 4 5 6 7
2 3
3 6 2 8 9 100
4 7 3 0 4 26
5 2 5 2 9 95
```

【样例 1 输出】

```
1 126
```

【数据范围与约定】

对于 40% 的数据， $1 \leq m \leq 3$ ， $Li_{min}, Co_{min}, Ni_{min}, Mn_{min}, Li_i, Co_i, Ni_i, Mn_i \leq 10$ 。

对于全部数据， $1 \leq m \leq 100$ ， $Li_{min}, Co_{min}, Ni_{min}, Mn_{min}, Li_i, Co_i, Ni_i, Mn_i \leq 30$ ，
保证答案小于 2147483647。

G . 服务划分 / Independent service

时间限制： 2.0 秒

【题目描述】

某电动汽车企业的电池服务网络中， n 个换电站以树形拓扑相连，每条连接线路有相应的输电成本。每个换电站有一定的电池需求 d_i 。企业需要在某些站点设立**服务中心**，每个站点由距离其最近的服务中心供应电池。随着运营的进行，换电站的需求量和线路的输电成本可能发生变化，企业需要动态调整服务区划分。

形式化地，给定一棵 n 个节点的带权树，节点 i 有权值 d_i 代表电池需求。树上的边 (u, v) 有权值 $w(u, v)$ 代表输电成本。两点间距离为路径上边权之和。

你需要处理 q 次操作，操作分为三种类型：

- **类型 1 — 修改需求**: 1 u val , 将节点 u 的需求量 d_u 修改为 val 。
- **类型 2 — 修改边权**: 2 u v w , 将树上边 (u, v) 的输电成本修改为 w 。
- **类型 3 — 查询服务量**: 3 k s_1 s_2 ... s_k , 指定 k 个服务中心站点 $s_1, s_2, \dots, s_k$ 。

对于树上的任意节点 u ，定义其**归属**为距离 u 最近的服务中心，若有多个距离并列最近则选择编号最小的。节点 u 的需求 d_u 计入该服务中心的服务总量。你需要输出 k 个整数，按 $s_1, s_2, \dots, s_k$ 的顺序给出每个服务中心的服务总量。

题目保证所有类型 3 操作的 k 之和不超过 3×10^5 。

【输入格式】

从标准输入读入数据。

第一行一个整数 n 。

第二行 n 个整数 $d_1, d_2, \dots, d_n$ 。

接下来 $n - 1$ 行，每行三个整数 u, v, w ，描述一条树边及其边权。保证 $1 \leq u, v \leq n$, $1 \leq w \leq 100$ 。

接下来一行一个整数 q 。

接下来 q 行，每行描述一次操作。每行第一个整数为操作类型 $op \in \{1, 2, 3\}$ ：

- 若 $op = 1$ ：后接两个整数 u, val ，表示将 d_u 修改为 val 。
- 若 $op = 2$ ：后接三个整数 u, v, w ，表示将边 (u, v) 的权值修改为 w 。保证 (u, v) 是树上的一条边。
- 若 $op = 3$ ：首先一个整数 k 表示服务中心数量，接着 k 个互不相同的整数 $s_1, s_2, \dots, s_k$ 。

【输出格式】

输出到标准输出。

对于每个类型 3 的操作，输出一行 k 个整数，依次表示 $s_1, s_2, \dots, s_k$ 的服务总量。

【样例 1 输入】

```
1 5
2 10 20 30 40 50
3 1 2 2
4 1 5 5
5 2 3 3
6 2 4 1
7 5
8 3 2 1 3
9 1 2 100
10 3 2 1 3
11 2 2 3 10
12 3 3 4 2 5
```

【样例 1 输出】

```
1 120 30
2 200 30
3 40 140 50
```

【样例 1 解释】

树的结构为：

```
1      1
2     / \
3    2   5
4   / \
5  3   4
```

边权：1 - 2 : 2, 2 - 3 : 3, 2 - 4 : 1, 1 - 5 : 5。初始需求： $d = [10, 20, 30, 40, 50]$ 。

操作 1（查询，服务中心：1, 3）：

- 节点 1 距离 1 为 0，距离 3 为 5 → 归属 1，贡献 10；
- 节点 2 距离 1 为 2，距离 3 为 3 → 归属 1，贡献 20；
- 节点 3 距离 1 为 5，距离 3 为 0 → 归属 3，贡献 30；
- 节点 4 距离 1 为 3，距离 3 为 4 → 归属 1，贡献 40；
- 节点 5 距离 1 为 5，距离 3 为 8 → 归属 1，贡献 50。

服务中心 1 服务总量 = $10 + 20 + 40 + 50 = 120$ ，服务中心 3 服务总量 = 30。

操作 2（修改需求）：将节点 2 的需求量修改为 100，新的 $d = [10, 100, 30, 40, 50]$ 。

操作 3（查询，服务中心：1, 3）：归属情况不变，服务中心 1 服务总量 = $10 + 100 + 40 + 50 = 200$ ，服务中心 3 服务总量 = 30。

操作 4（修改边权）：将边 (2, 3) 的权值修改为 10。

操作 5（查询，服务中心：4, 2, 5）：

- 节点 1 距离 4 = 3，距离 2 = 2，距离 5 = 5 $\rightarrow$ 归属 2，贡献 10；
- 节点 2 距离 4 = 1，距离 2 = 0，距离 5 = 7 $\rightarrow$ 归属 2，贡献 100；
- 节点 3 距离 4 = 11，距离 2 = 10，距离 5 = 17 $\rightarrow$ 归属 2，贡献 30；
- 节点 4 距离 4 = 0，距离 2 = 1，距离 5 = 6 $\rightarrow$ 归属 4，贡献 40；
- 节点 5 距离 4 = 6，距离 2 = 7，距离 5 = 0 $\rightarrow$ 归属 5，贡献 50。

输出：40, 140, 50。

【数据范围与约定】

对于 30% 的数据， $n \leq 5000$ ， $q \leq 300$ ，且仅包含类型 3 查询操作，不包含修改操作。 $1 \leq d_i, val, w \leq 100$ 。

对于 100% 的数据， $n, q \leq 3 \times 10^5$ ， $\sum k \leq 3 \times 10^5$ ，包含全部三种操作。 $1 \leq d_i, val, w \leq 100$ 。

H . 电池续航 / Journey endurance

时间限制： 1.0 秒

【题目描述】

在智能电动汽车的充电网络中， N 个充电站由单向行驶路段连接。一辆电动车需要完成恰好跨越 K 段路的旅程。电池容量必须不小于途经路段的**最大能量消耗值**，否则会在半路耗尽电量。充电站网络中存在自环与重边，需要仔细规划路线，使得所需的电池容量最小。

给定一个有向图，包含 N 个节点和 M 条有向边，每条边的能量消耗值为 w 。

有 Q 次询问，每次询问给出起点 s 和整数 K 。对于每个终点 $j \in [1, N]$ ，求从 s 出发，**恰好**经过 K 条边到达 j 的所有路径中，路径上**最大边权的最小值**。允许重复经过节点和边。若无法恰好 K 步到达 j ，输出 -1 。

图中可能存在**自环** $u \rightarrow u$ 和**重边**，即同一对有向节点间可能有多条同向边，权值可能不同。但是司机们都是聪明的，它们会在重边走最短的那条，不过，他们可以通过走自环留在起点。不能呆在原地不动，因为这是违章停车。

【输入格式】

从标准输入读入数据。

第一行两个整数 N, M ，表示充电站数量和路段数量。

接下来 M 行，每行三个整数 u, v, w ，描述一条从 u 到 v 的有向路段，能量消耗为 w 。可能存在自环和重边。

接下来一行一个整数 Q 。

接下来 Q 行，每行两个整数 s, K ，表示从 s 出发恰好走 K 步。

【输出格式】

输出到标准输出。

对于每个询问，输出一行 N 个整数，依次表示到达节点 $1, 2, \dots, N$ 所需的最小电池容量。若无法到达，输出 -1 。

【样例 1 输入】

```
1 3 6
2 1 2 5
3 2 1 2
4 2 3 8
5 1 3 100
```

```
6 1 1 10
7 2 1 1
8 2
9 1 1
10 1 3
```

【样例 1 输出】

```
1 10 5 100
2 10 5 10
```

【样例 1 解释】

网络中有 3 个充电站，6 条路段：

- $1 \rightarrow 2$ 消耗 5， $2 \rightarrow 1$ 消耗 2， $2 \rightarrow 3$ 消耗 8， $1 \rightarrow 3$ 消耗 100；
- $1 \rightarrow 1$ 消耗 10，是一条自环；
- 另一条 $2 \rightarrow 1$ 消耗 1，与已有的 $2 \rightarrow 1$ （消耗 2）形成重边。对重边只保留消耗最小的那条，即 1。

因此从各节点出发走 1 步的最优消耗为： $1 \rightarrow 1$ 需 10， $1 \rightarrow 2$ 需 5， $1 \rightarrow 3$ 需 100， $2 \rightarrow 1$ 需 1， $2 \rightarrow 3$ 需 8，其余方向无法直达。

询问 1 ($s = 1, K = 1$)：直接沿用上述 1 步消耗，到达节点 1,2,3 分别需容量 10,5,100。输出 **10 5 100**。

询问 2 ($s = 1, K = 3$)：

- 到 1： $1 \rightarrow 1 \rightarrow 1 \rightarrow 1$ 消耗 10， $1 \rightarrow 2 \rightarrow 1 \rightarrow 1$ 消耗 10，最小 10；
- 到 2： $1 \rightarrow 1 \rightarrow 1 \rightarrow 2$ 消耗 10， $1 \rightarrow 2 \rightarrow 1 \rightarrow 2$ 消耗 5，最小 5；
- 到 3： $1 \rightarrow 1 \rightarrow 2 \rightarrow 3$ 消耗 $\max(10, 5, 8) = 10$ 。

输出 **10 5 10**。

【数据范围与约定】

对于 30% 的数据， $N \leq 10$ ， $M \leq N^2$ ， $Q \leq 10$ ， $K \leq 100$ ， $1 \leq w \leq 10^9$ 。

对于 100% 的数据， $N \leq 100$ ， $M \leq N^2$ ， $Q \leq 1000$ ， $K \leq 10^9$ ， $1 \leq w \leq 10^9$ 。

I . 电池健康 / Keep health

时间限制： 1.0 秒

【题目描述】

某地有一座动力电池充换电站，负责管理电动汽车动力电池的入库、取用、归还与报废。电池的健康状态由充放电循环次数衡量，简称 SOH。请你编写程序，模拟该换电站的电池管理系统。其中，换电站支持四种操作：

1. **买进电池**：购买一颗新电池并存入仓库。电池拥有一个唯一的名字，由大写字母和数字组成，长度不超过 8 个字符，初始充放电循环次数为 0。
2. **取用电池**：从仓库中取出一颗电池供车辆使用。选取规则为：优先取充放电循环次数最少的电池；若有多颗并列，则取入库时间最早的电池。操作完成后输出被取出的电池的名字。
3. **归还电池**：某颗给定名字的电池被归还到仓库进行充电。充电会使该电池的充放电循环次数 +1。充电完成后，若该电池的充放电循环次数达到 K （即 $\geq K$ ），则该电池报废，**不再**放回仓库，并输出该电池的名字作为报废提示；否则，该电池重新入库，不输出任何内容。
4. **查询库存**：统计当前仓库中有多少颗电池，输出该数量。

最初仓库是空的。保证所有操作均合法：取用电池时仓库非空，归还的电池是当前已被取出的电池，买进的电池名字未被使用过，且所有电池名字均为长度不超过 8 的 ASCII 字符串（仅含大小写字母和数字）。

【输入格式】

从标准输入读入数据。

输入的第一行包含两个正整数 Q, K ，分别表示操作次数和电池报废的循环次数阈值。

接下来 Q 行，每行描述一个操作，格式如下：

- 1 <name>：买进一颗名字为 <name> 的新电池。
- 2：取用一颗电池。
- 3 <name>：归还名字为 <name> 的电池。
- 4：查询仓库中的电池数量。

【输出格式】

输出到标准输出。

对于操作 2（取用电池），输出被取出的电池的名字，占一行。

对于操作 3（归还电池），若电池报废，输出该电池的名字，占一行；否则不输出。

对于操作 4（查询库存），输出当前仓库中的电池数量，占一行。

【样例 1 输入】

```
1 13 3
2 1 B001
3 1 B002
4 2
5 3 B001
6 2
7 3 B002
8 2
9 3 B001
10 2
11 3 B002
12 2
13 3 B001
14 4
```

【样例 1 输出】

```
1 B001
2 B002
3 B001
4 B002
5 B001
6 B001
7 1
```

【样例 1 解释】

样例中 $K = 3$ ，共有 13 次操作，B001 被反复取用和归还，最终在第 12 次操作（第三次归还 B001）时循环次数达到 3 而报废。第 13 次查询显示仓库仅剩一颗电池（B002）。

【数据范围与约定】

对于 30% 的数据， $Q \leq 1000$ ， $K \leq 10$ 。

对于 100% 的数据， $Q \leq 2 \times 10^5$ ， $K \leq 10^5$ 。

J . 生产优化 / Line optimization

时间限制： 1.0 秒

【题目描述】

某电动汽车工厂拥有一条由多道工序组成的流水生产线。生产线可以抽象为一张有向无环图，每个节点代表一个生产状态，每条有向边代表一道工序，工序的耗时即为边的长度。工厂的工程师可以投入资源对若干道工序进行优化，将它们的耗时分别缩短。请帮助工程师选择最优的优化方案，使得从原材料（节点 1）到成品整车（节点 n ）的总耗时最小。

形式化地，给定一张有向无环图，节点编号为 1 到 n ，保证对所有边 (u, v) 均有 $u < v$ 。

图中有 m 条边，每条边由四个整数 u, v, w, c 描述，表示一条从 u 指向 v 的工序，原始耗时为 w ，优化后的耗时为 c ，满足 $0 < c < w$ 。

你可以选择最多 K 条边，将其中每条边的耗时从 w 改为 c 。求在此操作后，从节点 1 到节点 n 的最短路径的长度的最小值。

【输入格式】

从标准输入读入数据。

第一行包含三个正整数 n, m, K ，分别表示图的节点数、边数和最多可优化的边数。

接下来 m 行，每行包含四个正整数 u, v, w, c ，描述一条有向边及其优化前后的耗时。保证 $u < v$ ， $c < w$ 。

【输出格式】

输出到标准输出。

输出一行一个整数，表示优化最多 K 条边之后，从节点 1 到节点 n 的最短路径的最小可能长度。

【样例 1 输入】

```
1 4 5 2
2 1 2 5 2
3 1 3 6 3
4 2 3 1 1
5 2 4 8 3
6 3 4 4 1
```

【样例 1 输出】

1 4

【样例 1 解释】

不优化任何边时，从 1 到 4 的最短路径为 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ ，耗时 $5 + 1 + 4 = 10$ 。

优化 1 条边时，最优方案为优化 (1,2) 或 (3,4) 等，最小耗时 7。

优化 2 条边时，可以同时优化 (1,2) 和 (3,4)，路径 $1 \xrightarrow{2} 2 \xrightarrow{1} 3 \xrightarrow{1} 4$ 耗时 $2 + 1 + 1 = 4$ 。

故 $K = 2$ 时答案为 4。

【数据范围与约定】

对于 30% 的数据， $n \leq 2000$ ， $m \leq 10000$ ， $K = 1$ ， $1 \leq c < w \leq 10^4$ 。

对于 100% 的数据， $n \leq 2 \times 10^5$ ， $m \leq 5 \times 10^5$ ， $K \leq 100$ ， $1 \leq c < w \leq 10^4$ 。